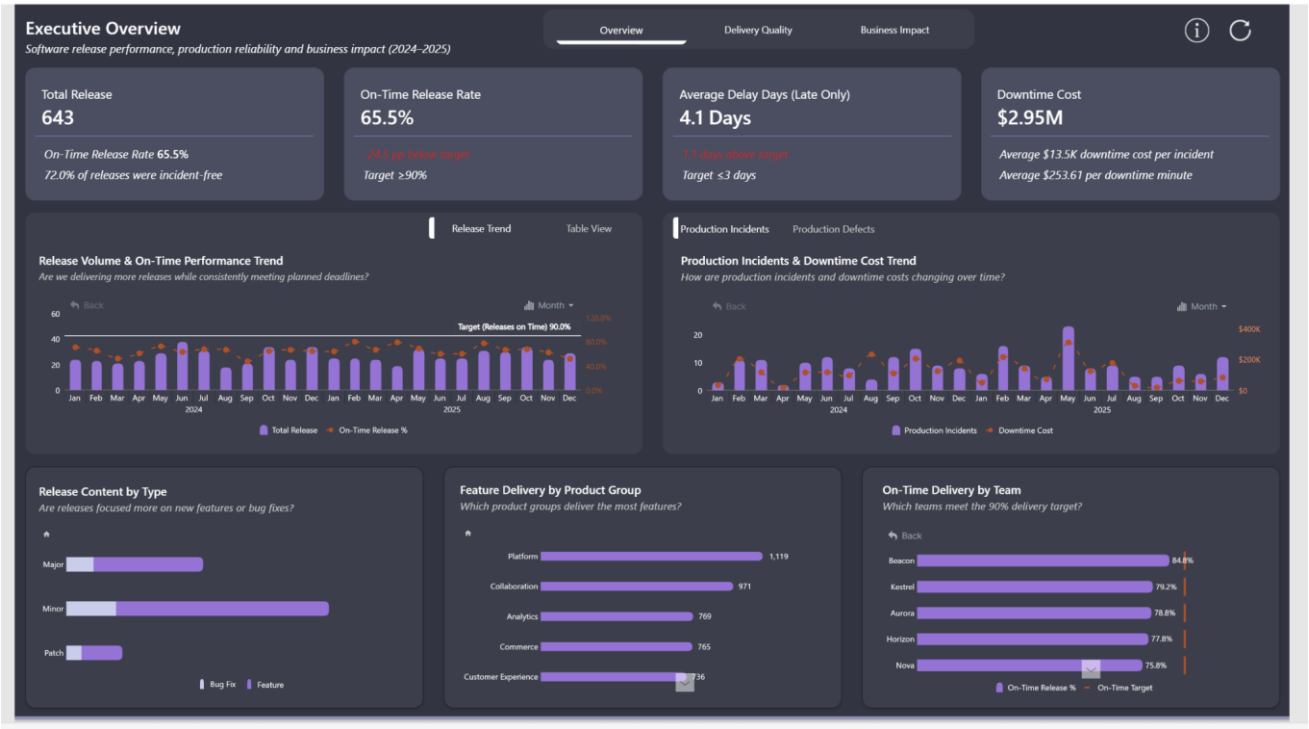
Software Delivery & Quality Analysis

Power BI Portfolio Case Study

Delivery predictability • release quality • production reliability • estimated business impact

NORTH-STAR QUESTION

How effectively are we delivering reliable software, and where do release failures create the greatest operational and financial impact?



The three-page Power BI report begins with an executive view of delivery, quality and business impact.

Analytical journey

Executive Overview → Delivery Quality → Production Downtime & Business Impact

REPORT	PERIOD	TOOLS	DESIGN
3 pages	2024–2025	Power BI · DAX	ZoomCharts · semantic model

View the interactive Power BI dashboard

Executive Summary

From release activity to a diagnosis of delivery reliability and production impact

Project purpose

This portfolio project evaluates software releases across 2024 and 2025 to assess whether delivery commitments are met, whether released software remains stable, and how production disruption translates into estimated downtime cost. The goal is to support management and delivery leaders with an evidence-led view of performance—not merely count releases.

ANALYTICAL JOURNEY

Are releases delivered reliably? → What delivery and testing outcomes accompany quality problems? → Where does the resulting disruption cost the most?

What I delivered

- A three-page interactive Power BI report: Executive Overview, Delivery Quality, and Production Downtime & Business Impact.
- A release-grain semantic model linked to product/module, team, release type and date, with item-level release contents.
- Reusable DAX measures, target comparisons, dynamic metric selectors, drill-downs and context-rich KPI reference labels.
- An analysis connecting missed deadlines, testing performance, production defects, incidents, downtime and estimated financial exposure.

Headline results

Measure	Result	Reference
Software releases	643	2024–2025
On-time release rate	65.5%	Target ≥90%
Avg delay, late releases	4.1 days	Target ≤3 days
Automation rate	73.1%	Target ≥75%
Production defects	738 · 1.15 per release	Target ≤2 / release
Production incidents	218 · 0.34 per release	Target ≤0.30 / release
Incident resolution	3.2 hours	Target ≤2 hours
Downtime / estimated cost	193.9 hours / $2.95M	Cost uses module rate assumptions

Portfolio takeaway

Releases are typically within the production-defect target, but delivery reliability, incident frequency and incident resolution fall short of their reference values. Financial exposure is concentrated: Core and BillingPro together account for approximately 72% of estimated downtime cost. The report therefore separates software quality from operational consequences instead of assuming that fewer defects always mean less business impact.

1. Business Context and Objectives

Treat delivery performance, production reliability and cost exposure as one management problem

Business context

The challenge dataset records planned and actual release dates, features, development and testing effort, pre-release and production defects, incidents, service downtime and cost-per-minute assumptions for product modules. These fields make it possible to assess delivery predictability and the operational consequences of releases within a common reporting period.

NORTH-STAR QUESTION

Are software updates being delivered on schedule and with acceptable quality—and which delivery outcomes represent the greatest business risk?

Analytical framework

Report page	Management question	Analytical focus
01 · Executive Overview	Are delivery and production reliability on track?	Releases, on-time performance, incidents, trends, teams and product mix.
02 · Delivery Quality	How do development and testing outcomes relate to released software quality?	Automation, failed tests, production defects, release type and product performance.
03 · Business Impact	Where is service disruption concentrated and which releases warrant attention?	Incidents, downtime duration, resolution, estimated cost and high-impact releases.

Success criteria

- Compare delivery and quality indicators to the dataset's explicit benchmarks, not isolated totals.
- Use rates per release to avoid conflating high release volumes with poor reliability.
- Connect engineering outcomes to module-specific downtime-cost exposure.
- Allow users to drill from trends or portfolio groupings to teams, products, release types and individual releases.
- Distinguish observed associations from causal explanations or confirmed root causes.

Decision perspective

The report moves from identifying performance gaps to examining possible contributing factors and then quantifying operational and financial exposure. Business impact is not isolated to the third page: it provides context in the overview and becomes the main prioritisation lens in the final investigation.

2. Dataset, Preparation and Semantic Model

Connecting release-level performance to product, team, calendar and release contents

Dataset scope and grain

Area	Role in the analysis
Release Performance Facts · 643 rows	One record per release: dates, development/testing, defects, incidents and downtime.
Release Items · 6,226 rows	One record per item: feature/bug-fix composition and functional categories.
Product · 36 modules	Product Group → Product → Module; contains merged downtime cost per minute.
Team · 15 teams	Named delivery teams for performance comparisons.
Release · 3 types	Major, Minor and Patch release classification.
Date and benchmark tables	Reporting time hierarchy and KPI performance thresholds.

Power BI semantic model

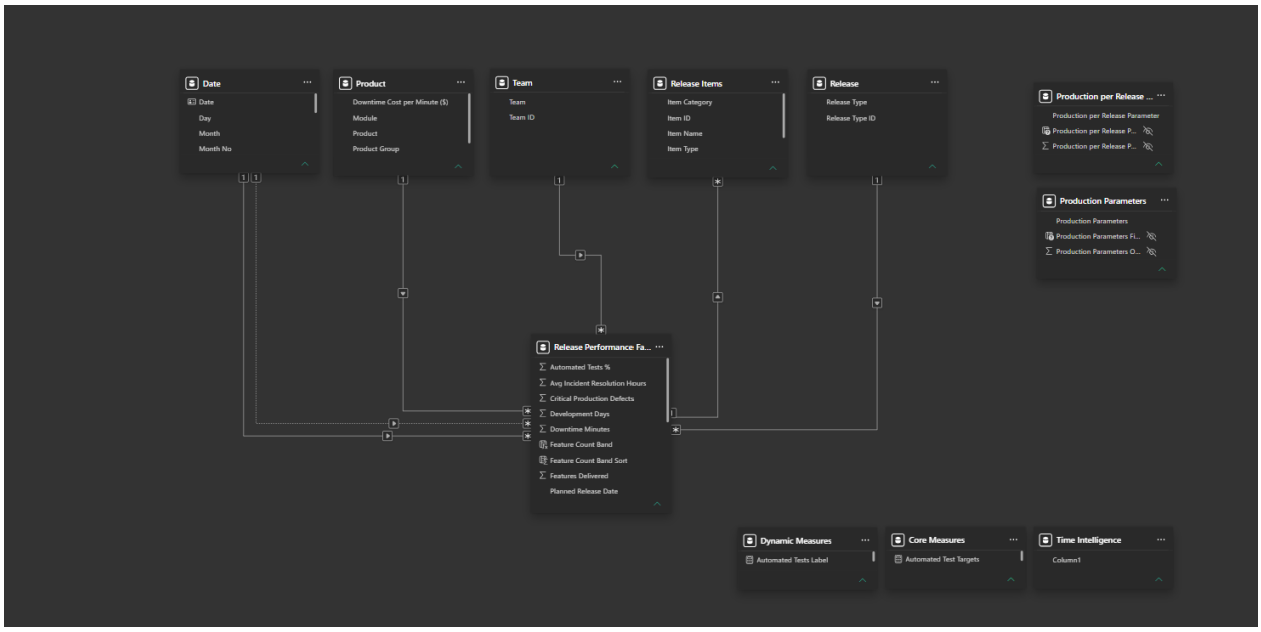


Figure 1. Release-level semantic model with dimension lookups, item-level details and disconnected measure helpers.

Key modelling decisions

- A single-release fact grain prevents production incidents and downtime from being multiplied by the number of items within a release.
- Downtime cost per minute is merged into Product at Product ID (module) level; cost is calculated across releases using their module rate.
- Relationships are principally single direction; item-category analyses of release-level KPIs use targeted TREATAS filters when needed.
- The date model supports analysis by actual release date, with the separate planned date retained for schedule variance analysis.

3. Measure Design and Analytical Logic

Build reusable rates and weighted metrics from base release-level measures

Core measure design

Measure	Definition / analytical role
On-Time Release %	On-time releases ÷ all releases; actual date on/before planned date.
Avg Delay Days (Late Only)	Average positive day difference among releases missing their planned date.
Automation %	Estimated automated tests ÷ tests executed; weighted by testing volume.
Test Failure Rate	Failed tests ÷ executed tests, aggregated in the current filter context.
Production Defects / Release	Production defects ÷ releases; comparable across product and release type.
Incidents / Release	Production incidents ÷ releases; reliability benchmark uses ≤0.30.
Avg Incident Resolution Hours	Incident-weighted average of each release's recorded resolution hours.
Estimated Downtime Cost	Sum of release downtime minutes × associated module cost-per-minute.

Selected reusable DAX patterns

Automation rate (test-volume weighted)

```
Automation % = DIVIDE ( [Automated Tests], [Tests Executed], 0 )
```

On-time release rate

```
On-Time Release % = DIVIDE ( [On-Time Releases], [Releases] )
```

Estimated downtime cost

```
Estimated Downtime Cost = SUMX ( 'Release Performance Facts', 'Release Performance Facts'[Downtime Minutes] *
RELATED ( Product[Downtime Cost per Minute ($)] ) )
```

Interpretation guardrails

- Source is a synthetic challenge dataset. Findings illustrate an analytical decision framework, not measured live-company outcomes.
- Estimated downtime cost is based on assumed $/minute rates; it is not an audited revenue loss or full financial impact.
- The facts are aggregated by release, so exact incident cause, chronology and severity cannot be reconstructed.
- Release composition categories can overlap across releases; their associated outcomes are not uniquely attributable to individual items.
- Relationships between testing, delay and defects are observational and cannot demonstrate causation.

4. Executive Overview

Are we delivering releases predictably without increasing operational exposure?

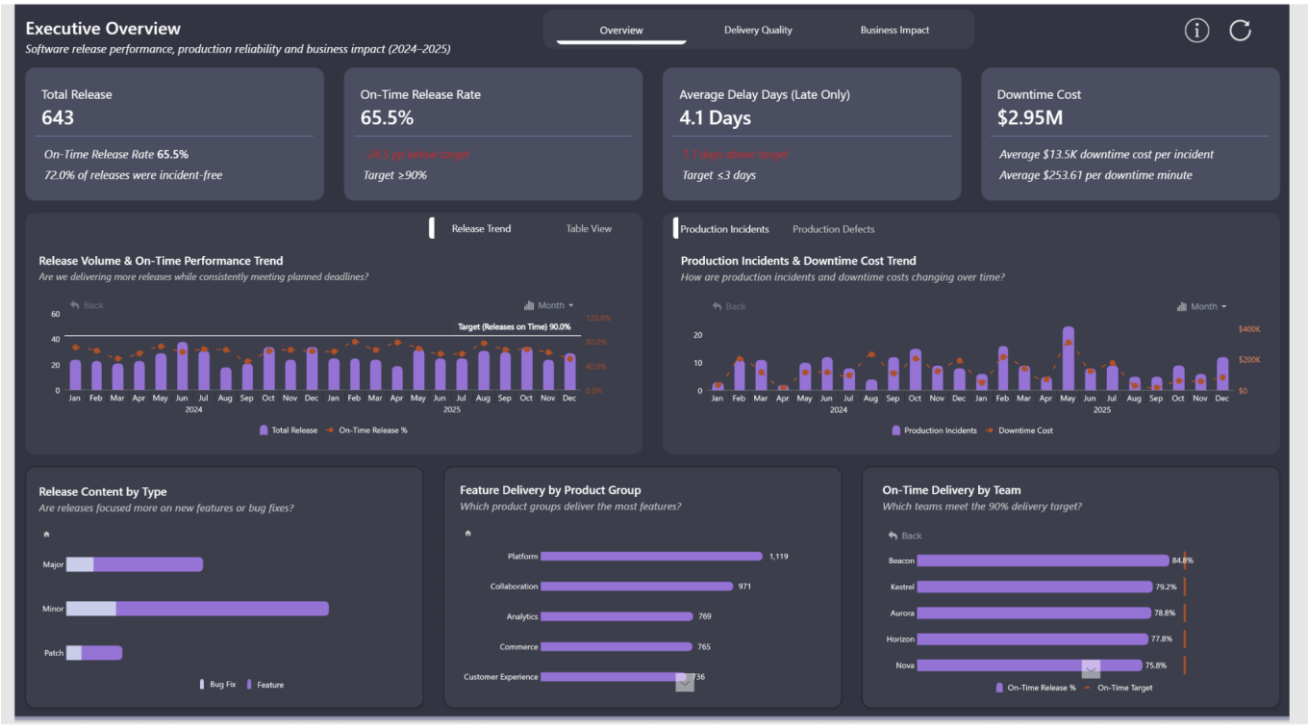


Figure 2. Overview: releases and punctuality, production incidents and costs, release content and team delivery performance.

What the dashboard reveals

- 643 releases were delivered; 65.5% met their planned date, 24.5 percentage points below the ≥90% benchmark.
- Among late releases, the average delay was 4.08 days versus the ≤3-day reference; schedule reliability is the clearest executive-level gap.
- Across 2024–2025, 218 production incidents were recorded and estimated downtime cost totalled $2.95M.
- Delivery performance varies materially across teams; even Beacon, at approximately 84.8% on time, remains below the 90% target.
- Platform delivered the most features (1,119), illustrating why delivery volume should be considered alongside reliability and cost exposure.
- On-time performance improved modestly from 64.1% (2024) to 66.9% (2025), while the incident rate moved from 0.33 to 0.35 per release.

DECISION VALUE

The overview identifies where outcomes diverge from targets, then invites investigation by delivery team, product and release type rather than implying that release volume alone measures success.

5. Delivery Quality

What does testing and release composition reveal about production defects?

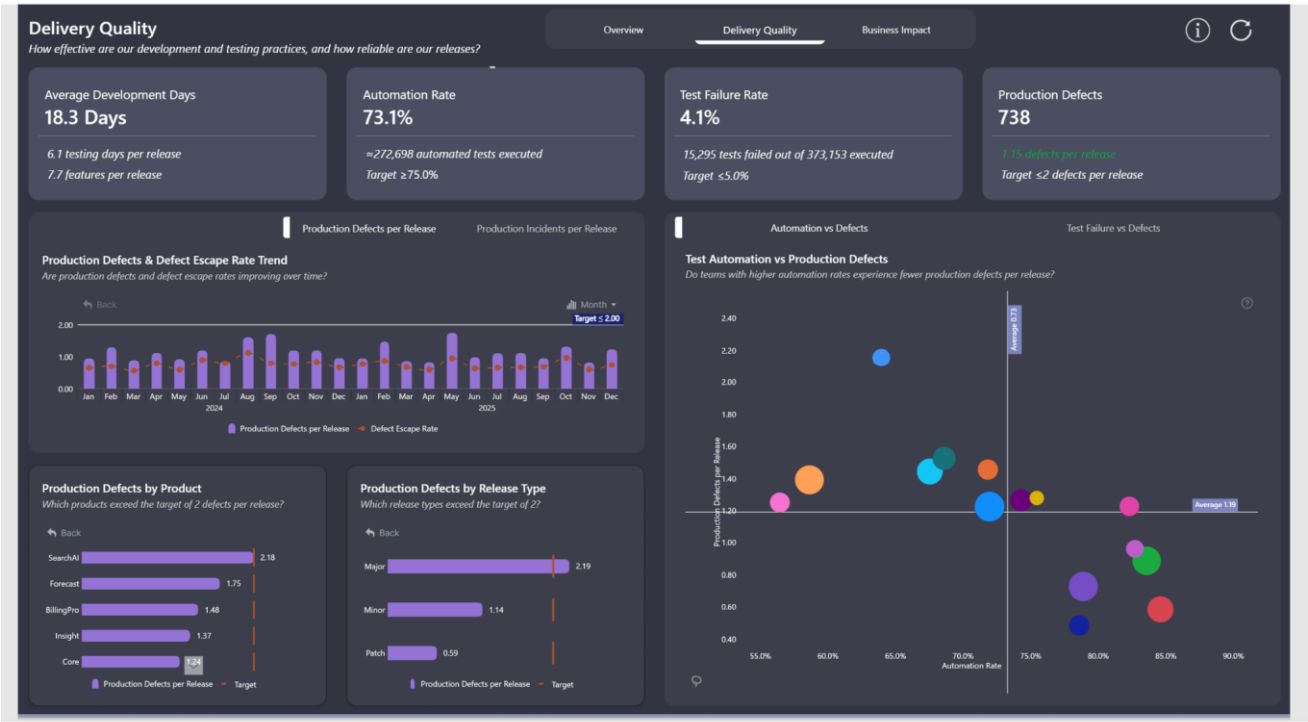


Figure 3. Delivery Quality: testing KPIs, defect trends, product and release-type comparisons, and automation/defects scatter.

What the dashboard reveals

- Average development effort is 18.3 days per release, with about 6.1 testing days; test failure rate is 4.1%, within the ≤5% benchmark.
- Test automation reaches 73.1%, slightly below the ≥75% benchmark, despite approximately 373,153 tests being executed.
- Production defects total 738 (1.15 per release). Major releases average 2.19 defects, exceeding the target of two; patch releases average only 0.59.
- SearchAI records 2.18 defects per release, above the threshold, while the dashboard's product comparison highlights variation in quality outcomes.
- Releases below 75% automation recorded 1.38 production defects per release versus 0.89 at or above 75%; this is an association, not a causal effect.

DECISION VALUE

Use release type and product context when evaluating testing maturity; one aggregate automation KPI can mask material differences in production outcomes.

6. Production Downtime & Business Impact

Which disruptions create the largest financial exposure, and what should be investigated first?

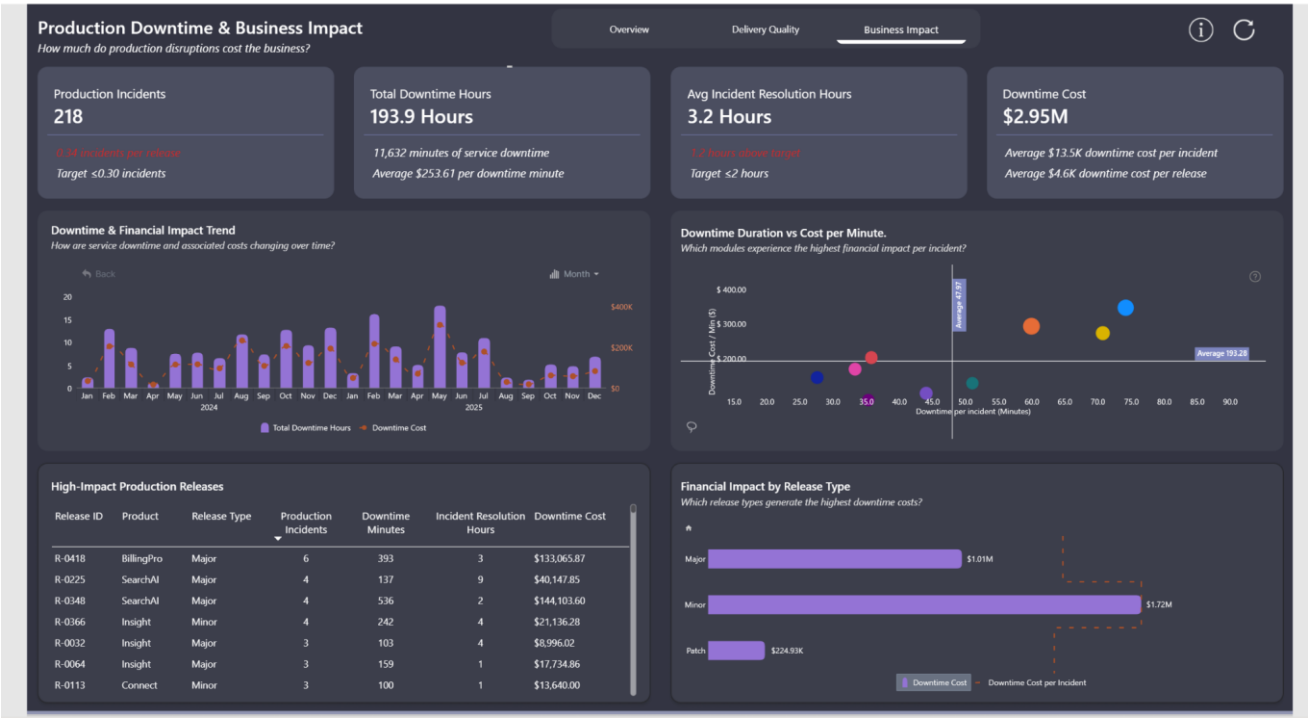


Figure 4. Business Impact: downtime/cost trend, duration vs cost-per-minute exposure, high-impact releases and release types.

What the dashboard reveals

- 218 incidents generated 11,632 minutes (193.9 hours) of service downtime and approximately $2.95M in estimated cost.
- The incident-weighted average resolution time is 3.2 hours—around 1.2 hours above the ≤2-hour benchmark.
- Core contributes approximately $1.14M and BillingPro $0.98M; combined, they account for around 72% of estimated downtime cost.
- Minor releases account for $1.72M (58% of the total) because of their collective exposure; major releases, however, have a much higher incident rate (0.85 incidents/release).
- The release detail table pinpoints costly exceptions. For example, R-0348 (SearchAI, Major) is associated with roughly $144.1K in estimated downtime cost.

DECISION VALUE

Prioritisation should consider incident frequency, downtime duration and the affected module's cost per minute—not incident count alone.

7. Recommendations

Evidence-led priorities for improving delivery reliability, quality and operational resilience

1. Restore delivery predictability where schedule gaps are largest.

On-time delivery is 65.5% against a 90% target, and late releases average 4.08 days over schedule. Review planning assumptions, dependency bottlenecks and approval queues in the weakest-performing teams; track on-time rate alongside late-only delay severity.

2. Apply proportionate quality gates to major releases.

Major releases record 2.19 production defects and 0.85 incidents per release, both above the relevant benchmarks. Use risk-based test plans, release-readiness checks and staged deployments for major changes before increasing throughput.

3. Close testing gaps without treating automation as a silver bullet.

Automation is 73.1% against the 75% target. Releases below that threshold show higher unadjusted defects/release (1.38 vs 0.89). Investigate whether test coverage, release type or complexity explains the difference and target automation to high-risk paths.

4. Prioritise Core and BillingPro for disruption prevention.

Core ($1.14M) and BillingPro ($0.98M) account for about 72% of estimated downtime cost. Drill to modules and releases to identify where observability, rollback capability, resilience testing and incident prevention could have greatest value.

5. Improve incident recovery against the two-hour benchmark.

Average incident resolution is approximately 3.2 hours versus a ≤2-hour target. Standardise incident runbooks, improve alert routing and use post-incident reviews to shorten restoration times; track downtime per incident alongside mean resolution time.

6. Manage the aggregate cost exposure of minor releases.

Minor releases account for about $1.72M of the $2.95M total even though major releases are riskier per release. Focus monitoring on cumulative cost by release type, not only average defect or incident intensity.